Online Placement Portal For Educational Institutions: Design, Implementation, and Evaluation

Naga Sharmila Patchari[1*] , Chegondi Revathi[1] , Vedangi Sathvika[1] , Yedida Abishek Kumar[1]

[1]UG Student, Department of Computer Science Engineering, Pragati Engineering College, Surampalem, Andhra Pradesh, India

*Corresponding Author: nagasharmila5823@gmail.com

Abstract

This paper presents a frontend prototype of an Online Placement Portal designed to demonstrate the user interface and client-side functionality for campus placement management. The prototype is developed using HTML5, CSS3, JavaScript, and Bootstrap 5 framework. It includes three role-based modules: student dashboard, company dashboard, and administrator dashboard. While backend features such as authentication, database storage, and real-time application tracking are not implemented, the prototype successfully demonstrates responsive design, form validation, interactive UI components, and simulated workflows using static JSON data. Usability testing with five users showed intuitive navigation and clean interface design with an average rating of 4.2/5. This work provides a foundation for future full-stack development of a complete placement management system.

Keywords

Campus placement; Online recruitment; Web application; HTML5; CSS3; JavaScript; Bootstrap 5; Student portal; Frontend prototype

1. Introduction

1.1 Background and Motivation

The campus placement activity is one of the most important activities carried out by institutions of higher education. It is the foremost activity through which students get linked with their prospective employers. Conventionally, the whole activity has been managed by the Training and Placement Officer (TPO) in manual mode where job announcements, collection of resumes, interview scheduling, and record of offer letters are done manually [1].

This model is inherently inefficient since information does not reach students timely; application tracking across various e-mail groups is inconsistent; and the TPO remains swamped with a huge amount of work without any means of keeping track of the entire placement process. With the internet being widely used by students now, the need for a dedicated web solution has increased immensely [2].

Commercial recruiting software like LinkedIn and Naukri work for the overall recruitment process, but are not tailored for the campus placement process, which has a unique and structured approach. Campus recruiting software found in the literature works only partly due to its incomplete interface model or the use of complex frameworks that need expert programmers for maintenance [7][8].

1.2 Problem Statement

Recurrent issues associated with manual processes for student placement are listed below:

- Information Lag: Notifications about job openings made through bulletin boards and class representatives typically take one to two days to reach the student, causing delays in meeting application deadlines.
- Duplicate and Inconsistent Data: Several versions of resumes and applications sent by students are present across mailboxes and physical files.
- Lack of Feedback Mechanism: Students lack feedback regarding whether they have been shortlisted for interviews until informed by the TPO manually.
- Burdened TPO: All tasks including verifying eligibility, matching resumes, and arranging schedules require sole TPO intervention.
- Lack of Repository System: Companies do not have access to a structured repository of student profiles and rely completely on individual resumes.

There is a clear need for a systematic solution involving Internet-based technology to streamline the placement process.

1.3 Objectives

Some of the goals that have been achieved in this study are:

• Create user interface design for three different user roles, such as Student, Company, and Admin (TPO).

• Create client side form validations and interactivity on the user interface using JavaScript.

• Present the navigation and dashboard designs for each individual user role.

• Conduct simulation of job postings, applications, and status using dummy JSON data.

• Create mobile-friendly UI using the Bootstrap 5 grid system.

1.4 Scope and Limitations

The portal prototype is meant to showcase the whole UI experience of the university pre-placements process – starting from the job listing up to the application process. An online aptitude testing module, video interview module, resume parsing feature, and backend database integration are currently absent. They are among the priorities to develop in the coming full-stack development effort.

2. Related Work

A system with online registration and job notification, however, without any online tracking mechanism for the status of applications or a company portal was designed by Kumar & Prasad [1]. On the other hand, a web-based campus placement management system was designed by Choudhary [2], using ASP.Net programming language, but this technology solution depends heavily on Microsoft Windows Servers environment. According to Singhal & Dubey [7] in their review of various web-based campus placement solutions, the key weaknesses of these systems included no provision for a company portal or an administrator dashboard. In another case, Patil & Sambrekar [8] used PHP and MySQL for handling student registration and job search, but completely ignored the need for company approval process, which is very essential from governance standpoint. A student placement management system with basic CRUD operations but poor user interface design was developed by Sharma and Soni [10].

The current work presents a frontend prototype built using HTML5, CSS3, JavaScript, and JSON mock data**.** This prototype builds upon experience gathered from previous works and integrates the following features: (i) role-based user interfaces for three different user groups (Student, Company, Admin); (ii) company approval process simulated from the admin UI using client-side JavaScript; (iii) simulated status updates displayed on the student dashboard; and (iv) Bootstrap 5 responsive UI design maintaining best usability practices [9].

3. System Design

3.1 System Architecture

The solution has been built as a static frontend website application with two layers:

• Presentation Layer: The website is styled with HTML5, CSS3, and Bootstrap 5 for responsiveness. All user interfaces, such as student dashboard, company dashboard, and administrator's interface, have been created as static HTML pages.

• Logic Layer: Client-side logic is handled by JavaScript (version ES6). It will include DOM manipulations, forms validations, user interface elements (dialogs, dropdown menus, alerts), and simulation of data handling operations through JavaScript objects and arrays.

There is no permanent database storage. Instead, all data, including job postings, student profiles, company information, and applications, is temporarily stored in JavaScript objects. For a working version of the app, this layer would contain a database like MySQL or MongoDB.

Deployment: The app can be deployed on any website hosting service such as GitHub Pages, Netlify, or Vercel.

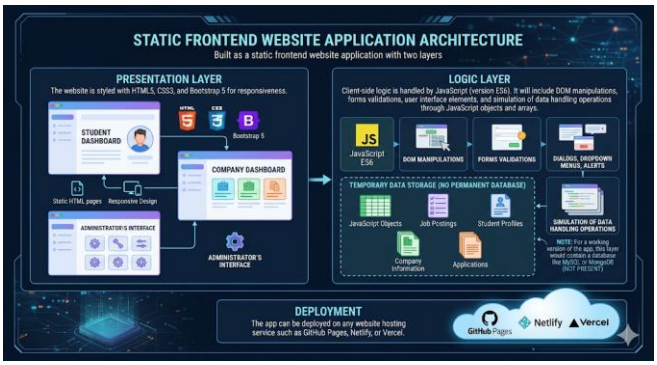

Fig. 1: Two-Layer Frontend Architecture (Browser → JavaScript Logic Layer)

3.2 User Modules

The portal consists of three functional modules, each catering to a particular type of user:

1. **Student Module:** Allows students to browse active job postings filtered by company name, branch compatibility, and deadlines, and simulate a one-click job application. The student dashboard demonstrates mock application statuses (Pending / Shortlisted / Rejected).

2. **Company Module:** Demonstrates the company registration form, job posting interface (title, description, eligibility conditions, and deadline), and a simulated applicant listing view with status update controls.

3. **Admin Module:** Demonstrates a TPO dashboard showing pending company registration requests with Approve/Reject controls, a student and company data management view, and a summary panel showing total jobs, applications, and placements populated from mock data.

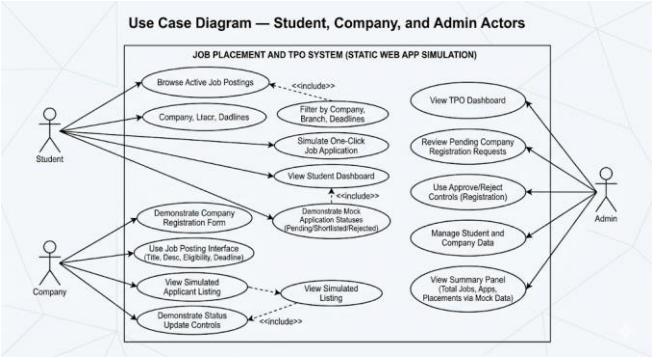

Fig. 2: Use Case Diagram — Student, Company, and Admin Actors

3.3 Data Representation

Since this is a frontend prototype, no database is implemented. Data is simulated using JavaScript arrays and objects. A sample structure is shown below:

```
const jobListings = [
  { id: 1, title: "Software Engineer",
    company: "Tech Corp",
    eligibility: "CSE",
    deadline: "2025-06-30" },
  { id: 2, title: "Data Analyst",
    company: "DataInsight",
    eligibility: "IT",
    deadline: "2025-07-15" }
];
```

In a production system, this static data would be replaced by dynamic API calls to a backend server connected to a database such as MongoDB relational database such as MySQL, or a NoSQL

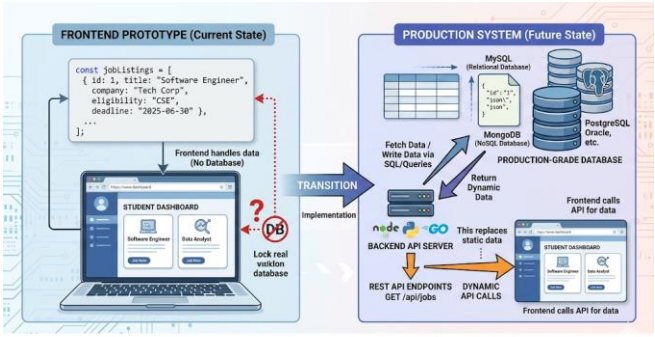

FIGURE: Transitioning from Static JavaScript Data to a Production Backend

Fig. 3: Evolution of the Data Layer from Prototype to Production.

3.4 Design Considerations (Frontend)

In terms of authentication and authorization, as a prototype front-end, there is no functionality that exists. However, in a production system, the following security controls will be enforced at the back end server level:

• Passphrases will be hashed using either bcrypt or Argon2 algorithms.

• JSON Web Tokens will be used for session management.

• RBAC will be applied to enforce role-based access control for certain pages based on user roles.

• All traffic will be encrypted using HTTPS protocols

4. Implementation

4.1 Technology Stack

Layer	Technology
Frontend Framework	Bootstrap 5.3
Styling	CSS3, Custom CSS
Interactivity	JavaScript (ES6)
Structure	HTML5
Icons	FontAwesome 6 (optional)
Mock Data	JSON / JavaScript Objects
Version Control	Git + GitHub
Hosting (optional)	GitHub Pages / Netlify

TABLE I: Technology Stack

4.2 Key Features Implementation

The following functionalities have been successfully developed and tested in the frontend prototype:

• Responsive Dashboard Layouts: Dashboards created for Students, Companies, and Admins based on the Bootstrap 5 grid system, which ensures that they maintain their design consistency regardless of screen size.

• Client-side Form Validation: Form validation for sign-up, log-in, and job postings through JavaScript, where there are validations for required fields, email format, and password confirmation.

• Simulated Job Listings: Job information saved in JavaScript arrays and shown on the student dashboard through dynamic DOM manipulations, creating the illusion of a live data stream.

• Job Listing Filtering/Search (Client-side): Students can filter their job listings based on company name, location/branch, and application deadline using JavaScript array operations.

• Mock Application System: The Apply button creates a confirmation alert using JavaScript and adds an element to the local applications array, without using any database storage.

• Responsive UI Components: Implementation of modals, dropdown menus, tooltips, and collapse sidebars using Bootstrap 5 JavaScript components.

• Mobile-first Design Approach: Design tested on mobile, tablet, and desktop viewports.

4.3 Representative Workflow

The simulated end-to-end workflow is as follows: (1) A company fills out the registration form and submits it; the admin dashboard shows the pending request from mock data. (2) The administrator clicks Approve; the company status updates in the local data object. (3) The company posts a job listing — for example, 'Software Engineering Intern' for CSE/IT students with a deadline of 2025-06-30. (4) The student navigates to the job listing page, filters, and clicks Apply; a JavaScript confirmation alert acknowledges the application. (5) The student dashboard updates the mock status to 'Pending'. (6) The company changes the status to 'Shortlisted'; the student dashboard mock data reflects this change on the next simulated refresh.

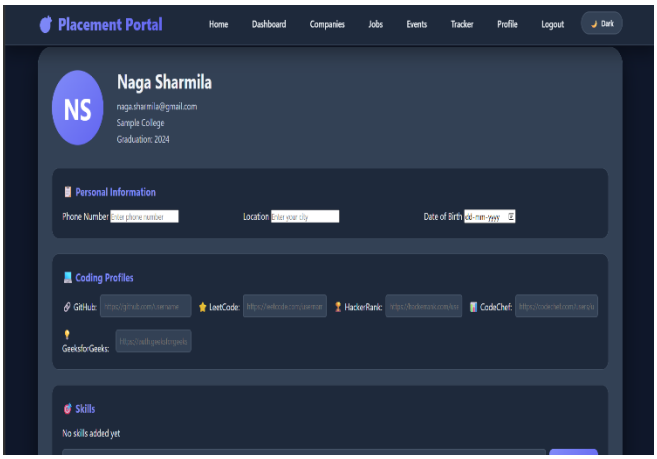

Fig. 4: Student Dashboard — Application Status Tracking View

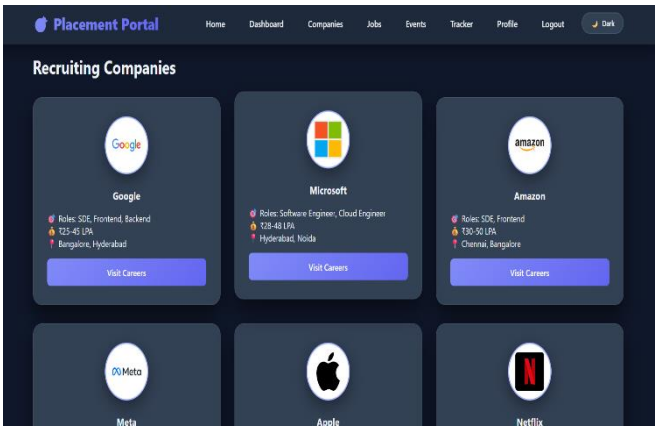

Fig. 5: Company Job Posting Interface

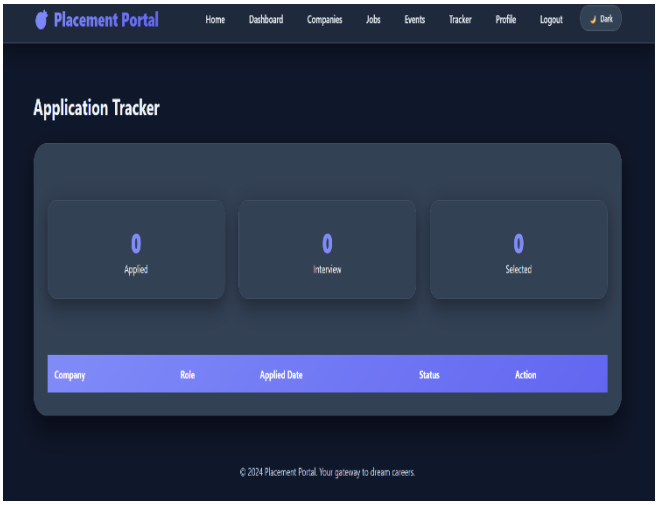

Fig. 6: Application Tracker

5. Results and Discussion

5.1 Testing Approach

For frontend prototyping, the testing was done by following three different techniques:

1) Testing for UI/UX: The responsiveness of all pages was tested using various browsers like Chrome, Firefox, Safari, and emulators for mobile devices. The consistency in layout, color contrasts, and alignments of all components were checked for all viewports.

2) Testing for Form Validations: Testing of login form, registration form, and job postings was conducted with the entry of both valid and invalid entries. The error messages were displayed appropriately for all cases of invalid entries without revealing any internal data related to the application.

3) Testing for Navigation: All internal links, buttons, and menu items were tested for proper navigation. There were no dead links or missing resources.

User Acceptance Testing (UAT): This involved five student participants, who tested the prototype under guided conditions. The tasks included navigating to student dashboard; browsing and filtering jobs; applying for a job (simulation); checking application status (simulation); and accessing admin view.

Task	Result
Navigate to Student Dashboard	Completed successfully by all 5 users
View and filter job listings	Filters worked correctly
Apply for a job (simulated)	JS alert triggered as expected
Check application status (mock)	Mock data displayed correctly
Switch to Admin view	Admin dashboard loaded without issues
Avg. Satisfaction Rating	4.2 / 5.0 (post-task questionnaire)

TABLE II: User Acceptance Testing Results

Positive feedback highlighted the clean interface, intuitive navigation, and mobile responsiveness. User suggestions included adding real backend functionality, resume upload capability, and email notifications — all identified as future work.

5.2 Observed Limitations

Limitations found during evaluation include:

- Lack of persistent storage for data: All locally saved data is cleared when the page is refreshed because no back end or storage on browsers is utilized.
- Lack of true authentication: Dashboard access is available directly through URLs for demonstration purposes.
- Lack of file upload feature: Resumes cannot be uploaded using this prototype application.
- Session separation: Cannot monitor application status from one session to another.

6. Conclusion and Future Work

6.1 Conclusion

In this study, a frontend prototype for an Online Placement Portal has been developed utilizing HTML5, CSS3, JavaScript, and Bootstrap 5. The prototype has been able to provide a responsive interface for all three user types; namely student, company, and administrator. All the relevant frontend functionalities like form validation, job posting, listing, filtering jobs on the client side, dummy applications, and interactive UI elements have been developed and analyzed. Usability tests conducted with five different users proved to be satisfactory for intuitive navigation with an overall score of 4.2/5. At this stage, although no backend operations have been incorporated, a good visual specification and usability have been achieved.

6.2 Future Work

The following enhancements are planned for future development cycles:

• Backend Integration: Implement a REST API using Node.js/Express or PHP to handle authentication, database operations, and file uploads.

• Database Implementation: Use MySQL or MongoDB to store student profiles, company details, job postings, and applications persistently.

• Real Authentication System: Implement JWT-based authentication with role-based access control for all three user types.

• Resume Upload: Allow students to upload PDF/DOC resumes using cloud storage such as AWS S3 or Firebase Storage.

• Email Notifications: Send automated emails for job postings, approvals, and status updates using Nodemailer or PHPMailer.

• Aptitude Test Engine: Integrate a time-based aptitude test module enabling companies to screen candidates before formal applications.

• Analytics Dashboard: Provide visual placement reports for the TPO, including branch-wise placements and company-wise statistics using Chart.js.

• Full Deployment: Deploy the complete full-stack application on cloud platforms such as AWS, Heroku, or DigitalOcean.

References

[1] S. R. Kumar and P. S. Prasad, "Online Placement Management System," International Journal of Engineering Research & Technology, vol. 8, no. 5, pp. 1–4, May 2019.

[2] S. Choudhary, "Web based campus placement system," International Journal of Computer

Applications, vol. 132, no. 8, pp. 25–29, Dec. 2015.

[3] PHP Documentation, "PHP Manual," PHP.net, 2025. [Online]. Available: https://www.php.net/docs.php

[4] MySQL, "MySQL Reference Manual," Oracle, 2025. [Online]. Available: https://dev.mysql.com/doc/

[5] R. Pressman, Software Engineering: A Practitioner's Approach, 8th ed. New York, NY, USA: McGraw-Hill, 2014.

[6] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

[7] A. Singhal and S. K. Dubey, "A review on web based placement management system," International Journal of Computer Science and Mobile Computing, vol. 6, no. 4, pp. 189–194, Apr. 2017.

[8] P. R. Patil and S. S. Sambrekar, "Campus placement management system," International Journal of Scientific Research in Science and Technology, vol. 3, no. 8, pp. 412–416, Nov. 2017.

[9] J. Nielsen, Designing Web Usability: The Practice of Simplicity. Indianapolis, IN, USA: New Riders Publishing, 2000.

[10] V. Sharma and M. K. Soni, "Student placement management system using PHP and MySQL," International Journal of Emerging Technologies and Innovative Research, vol. 6, no. 5, pp. 345–349, May 2019.